Beyond Counting Tokens and Word2Vec: Introduction to Transformers and BERT

MSIS 521
Professor Hossein Ghasemkhani

FOSTER SCHOOL OF BUSINESS
UNIVERSITY OF WASHINGTON

1

Agenda

- Why LLMs matter for digital marketing
- From classical NLP to transformers
- What is BERT and how does it work?
- Hands-on: Using BERT in Python for text analysis
- Discussion: When to use BERT vs. other LLMs

FOSTER SCHOOL OF BUSINESS

2

2

Why LLMs Matter in Digital Marketing?

- Marketers work with massive amounts of text and media
- LLMs can help with:
  - Content generation (ads, emails, landing pages)
  - Customer insights from reviews and social media
  - Search and recommendations
  - Chatbots and customer support
- Goal today: understand the basics, and run BERT on real text in Python

FOSTER SCHOOL OF BUSINESS

3

3

From Bag-of-Words to Deep NLP

- Traditional approaches:
  - Bag-of-words, TF–IDF, n-grams
  - Each word is independent; order and context is mostly ignored
- Word embeddings (such as Word2Vec):
  - Dense vector representations that capture similarity
  - Still mostly one vector per word, regardless of context
- Modern models: contextual embeddings (BERT, GPT)

FOSTER SCHOOL OF BUSINESS

4

4

What Is a Language Model?

- A model that assigns probabilities to sequences of words
- Learns patterns from large text corpora
- Can be used to:
  - Predict the next word
  - Fill in missing words
  - Generate or classify text
- Large Language Models (LLMs) = very large language models with vast parameters trained on massive datasets (like Common Corpus)
  - From millions of parameters (BERT) to billions (GPT-4)

FOSTER SCHOOL OF BUSINESS

5

5

From RNNs to Transformers

- Early deep NLP: Recurrent Neural Networks (RNNs), LSTMs
- Limitations: slow, hard to parallelize, struggle with long-range dependencies
- Transformers (Vaswani et al., 2017):
  - Use only self-attention instead of recurrence
  - Enable parallel training on GPUs
  - Scale to very large models (LLMs)

FOSTER SCHOOL OF BUSINESS

6

6

Key Idea: Self-Attention

- Each word can "attend" to other words in the sentence.
- Model learns which words are important for understanding a given word
- Allows capturing long-range dependencies (e.g., pronouns, context)

Videos to learn more:
History and evolution of Transformers (from Y Combinator)
How to build a GPT (by Andrej Karpathy)

FOSTER SCHOOL OF BUSINESS

7

7

Transformer Architectures

- Encoder–decoder (e.g., original Transformer, many translation models)
- Decoder-only (e.g., GPT-style LLMs)
- Encoder-only (e.g., BERT)
- BERT is mainly used for:
  - Classification (sentiment, topic, intent)
  - Similarity / retrieval
  - Token-level tasks such as NER (Named Entity Recognition)

FOSTER SCHOOL OF BUSINESS

8

8

What Is BERT?

- BERT = Bidirectional Encoder Representations from Transformers
- Trained to understand context from both left and right
- Pre-training tasks:
  - Masked Language Modeling (MLM): predict masked words
  - Next Sentence Prediction (NSP): predict if sentence B follows sentence A
- We usually use pre-trained BERT and then:
  - Fine-tune on a downstream task
  - Or use it as a feature extractor (embeddings)

FOSTER SCHOOL OF BUSINESS

9

Why BERT for Marketing Analytics?

- Can understand nuanced language in:
  - Customer reviews and survey responses
  - Social media comments and posts
  - Support tickets and chat logs
- Useful tasks:
  - Sentiment and emotion detection
  - Topic and intent classification
  - Clustering and similarity of customer feedback

FOSTER SCHOOL OF BUSINESS

10

Pre-Trained vs Fine-Tuned BERT

- Pre-trained BERT models learn general language patterns
- Fine-tuning = training the model a bit more on a specific task
- In practice, we often use:
  - Pre-fine-tuned models (e.g., sentiment models)
  - Or fine-tune briefly on our own labeled data
- Today: we'll use an off-the-shelf model via Hugging Face

FOSTER SCHOOL OF BUSINESS

11

Hands-On Goals for Today

- Set up a BERT-based text classifier in Python
- Run sentiment analysis using BERT on a small set of marketing texts (reviews)
- Additional applications of BERT (to review outside the class):
  - Embedding and clustering
  - Sentiment analysis to identify negative and positive sentiments

FOSTER SCHOOL OF BUSINESS

12

Setting up BERT in Python

13

13

Environment Setup (Python)

- We will use a Jupyter Notebook on Google Colab:

```
!pip install transformers torch --quiet

import pandas as pd
from transformers import pipeline
```

14

Loading a Pre-Trained Sentiment Pipeline

```
sentiment_model = pipeline(
    task="sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment"
)

# Quick test
sentiment_model("This phone case is simply incredible!")
```

FOSTER SCHOOL OF BUSINESS

15

Running BERT on Example Text

```
reviews = [
    "Shipping was super fast and customer support was helpful.",
    "The product stopped working after only two weeks. Very disappointed.",
    "It's okay, not great but not terrible either."
]

results = sentiment_model(reviews)

for text, result in zip(docs, results):
    print(f"TEXT: {text}\nPREDICTION: {result}\n")
```

FOSTER SCHOOL OF BUSINESS

16

Sentiment Analysis of Online Product Reviews

17

17

Applying BERT to a spreadsheet of Reviews

```
# videogame_reviews.csv has a column 'review_text'
import pandas as pd
df = pd.read_csv("videogame_reviews.csv")

# For a quick demo, let's sample a subset
sample_df = df.sample(50, random_state=12)

sample_df["bert_sentiment"] = sample_df["review_text"].apply(
    lambda x: sentiment_model(str(x))[0]["label"]
)

sample_df.head()
```

18

Aggregating Sentiment Results

```
sentiment_counts = sample_df["bert_sentiment"].value_counts().sort_index()
print(sentiment_counts)

# Convert counts to percentages
sentiment_pct = sentiment_counts / sentiment_counts.sum() * 100
print(sentiment_pct)
```

FOSTER SCHOOL OF BUSINESS

19

Visualizing Sentiment Distribution

```
import matplotlib.pyplot as plt

sentiment_counts.plot(kind="pie")
plt.xlabel("Predicted Sentiment (Stars)")
plt.title("Review Sentiment Distribution (BERT)")
plt.show()
```

FOSTER SCHOOL OF BUSINESS

20

When to Use BERT vs. Other LLMs

- BERT / encoder models:
  - Great for classification, similarity, retrieval, clustering
  - Smaller, often faster and cheaper to run than giant LLMs
- Decoder-only LLMs (e.g., GPT-style):
  - Great for generation (copy writing, ideation, chatbots)
  - Can also do classification via prompting, but may be overkill
- In practice:
  - We may end up combining them in a system

FOSTER SCHOOL OF BUSINESS

21

Limitations & Responsible Use

- Models can be biased or wrong
- Need to validate models on your own data
- Consider privacy and terms of service when sending text to APIs
- Keep a human in the loop for important decisions

FOSTER SCHOOL OF BUSINESS

22

BERT for Embedding and Clustering

23

23

Using BERT as an Embedding Model

```
from transformers import AutoTokenizer, AutoModel

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

def get_cls_embedding(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )
    with torch.no_grad():
        outputs = model(**inputs)
    # CLS token embedding is at position 0
    cls_embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()
    return cls_embedding
```

24

Clustering Reviews with BERT Embeddings (Conceptual)

```
from sklearn.cluster import KMeans
Import torch
import numpy as np

# Take a small subset of reviews
texts = sample_df['review_text'].tolist()

embeddings = np.vstack([get_cls_embedding(t) for t in texts])

kmeans = KMeans(n_clusters=3, random_state=12)
labels = kmeans.fit_predict(embeddings)

for text, label in zip(texts, labels):
    print(f"Cluster {label}: {text[:80]}...")
```

FOSTER SCHOOL OF BUSINESS

25

Sentiment Analysis to Identify Negative and Positive Sentiments

26

FOSTER SCHOOL OF BUSINESS

26

Continuous Sentiment with BERT

```
from transformers import pipeline

# BERT-family sentiment model (binary: POSITIVE / NEGATIVE)
sentiment_model = pipeline(
    task="sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

def bert_signed_score(text):
    out = sentiment_model(text, truncation=True)[0]
    label = out["label"].upper()      # 'POSITIVE' or 'NEGATIVE'
    score = out["score"]              # confidence in predicted label (0 to 1)
    if label.startswith("NEG"):
        return -score                 # map NEGATIVE to [-1, 0)
    else:
        return score                  # map POSITIVE to (0, 1]

text = "Shipping was super fast and the support team was helpful."
print(text)
print("Sentiment score:", bert_signed_score(text))  # in [-1, +1]
```

FOSTER SCHOOL OF BUSINESS

27

Applying Continuous Sentiment to Reviews

```
# Apply BERT-based continuous sentiment to a DataFrame of reviews
def sentiment_score(text):
    return bert_signed_score(str(text))

sample_df["sentiment_score"] = sample_df["review_text"].apply(sentiment_score)

sample_df.head()
```

FOSTER SCHOOL OF BUSINESS

28

Aggregating Continuous Sentiment Scores

```
# Aggregate by time, product, or campaign (example: by 'product_id')
product_sentiment = (
    sample_df
    .groupby("product_id")["sentiment_score"]
    .mean()
    .sort_values()
    )
print(product_sentiment.head(10))

# Or simply look at the distribution overall
import matplotlib.pyplot as plt

sample_df["sentiment_score"].hist(bins=10)
plt.xlabel("Sentiment score (-1 to +1)")
plt.ylabel("Number of reviews")
plt.title("Distribution of Continuous Sentiment Scores (BERT-based)")
plt.show()
```

FOSTER SCHOOL OF BUSINESS

29